

Học Sâu (Deep Learning)

Chương 5: Nền tảng của Học máy (Fundamentals of ML)

Đại học Đông Á

August 14, 2026

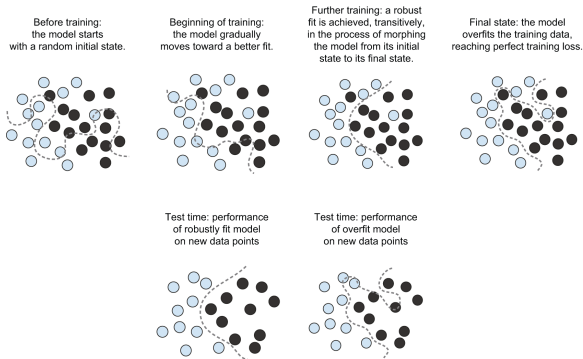
Nội dung chương

1. Tối ưu hóa (Optimization) và Khái quát hóa (Generalization)
2. Tác động của Dữ liệu Nhiễu (Noisy Data)
3. Các phương pháp Đánh giá Mô hình
4. Dung lượng Mô hình (Model Capacity)
5. Các Phương pháp Regularization
6. Giả thuyết Đa tạp (The Manifold Hypothesis)
7. Tổng kết

- **Tối ưu hóa (Optimization):** Quá trình điều chỉnh mô hình để đạt hiệu suất tốt nhất trên tập dữ liệu huấn luyện (*Training data*).
- **Khái quát hóa (Generalization):** Khả năng mô hình dự đoán chính xác trên dữ liệu hoàn toàn mới chưa từng thấy (*Test data*).
- **Mục tiêu:** Đạt được khả năng Generalization cao nhất. Tuy nhiên, thuật toán Gradient Descent chỉ có thể trực tiếp tác động vào quá trình Optimization.
- **Mâu thuẫn:** Sau một thời gian huấn luyện nhất định, Optimization càng tốt thì Generalization lại càng đi xuống.

Hiện tượng Underfitting và Overfitting

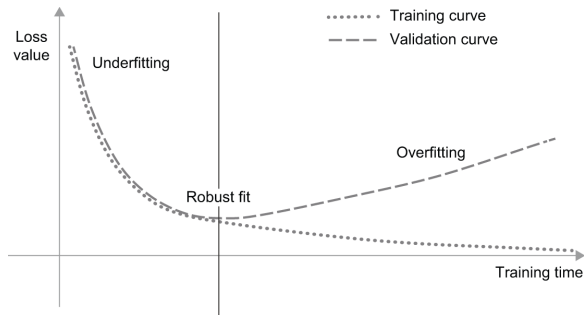
- **Underfitting (Chưa khớp):** Mô hình quá đơn giản, không học được quy luật trong dữ liệu. (Ví dụ: Dùng đường thẳng tuyến tính để khớp dữ liệu phân bố parabol).
- **Robust Fit (Khớp chuẩn):** Mô hình học được xu hướng thực sự của dữ liệu.
- **Overfitting (Quá khớp):** Mô hình quá phức tạp, học thuộc lòng cả các "nhiều" (noise) của dữ liệu huấn luyện, khiến đường biên quyết định cong queo, mất đi tính tổng quát.



Hình 5.1: Mô phỏng Underfitting, Robust Fit và Overfitting

Vòng đời điển hình của mô hình (Universal Overfitting)

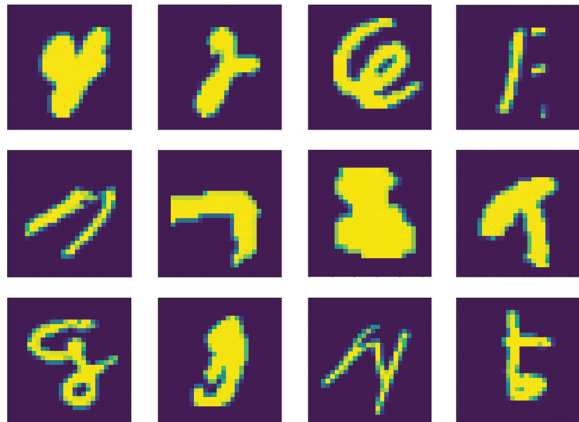
- Ở giai đoạn đầu, Loss trên tập Train và tập Validation cùng giảm, các tham số đang liên tục điều chỉnh đúng hướng → Giai đoạn Underfitting.
- Sau một số epoch nhất định, Validation Loss bắt đầu chứng lại và tăng vọt trở lại, dù Training Loss vẫn tiếp tục giảm tiệm cận 0 → **Giai đoạn Overfitting**.



Hình 5.2: Hành vi Overfitting kinh điển

Dữ liệu không hợp lệ (Invalid Inputs)

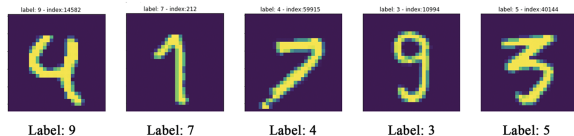
- Trong thực tế, dữ liệu thường không hoàn hảo, chứa nhiều tạp âm, nhòe nhoẹt.
- Ví dụ: Tập dữ liệu ảnh số viết tay MNIST có chứa những hình ảnh bị lỗi, mờ đến mức mắt người cũng không thể nhận diện được.
- Nếu mạng nơ-ron có đủ dung lượng, nó sẽ cố gắng "học vẹt" những hình nhiễu này bằng cách dùng một phần tài nguyên mạng ghi nhớ chúng. Điều này làm suy giảm tính khái quát.



Hình 5.3: Các ảnh bị nhiễu nặng trong tập MNIST

Dữ liệu bị gán nhãn sai (Mislabeled Data)

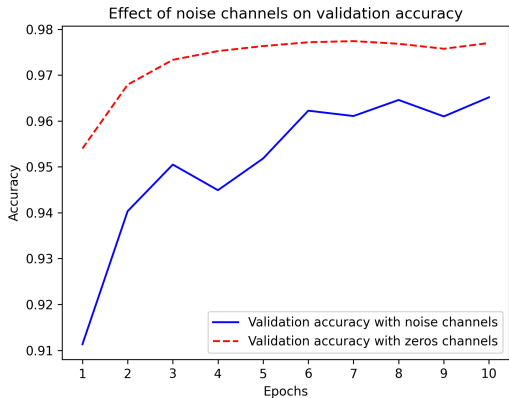
- Do sai sót chủ quan của con người (Human Error), một số dữ liệu có thể bị gán nhãn (label) hoàn toàn sai.
- Ví dụ: Rõ ràng là số 4 nhưng lại gán nhãn là số 9 trong tập MNIST.
- Nếu mô hình cố gắng uốn cong ranh giới quyết định (decision boundary) để "chữa" lại những nhãn sai này, nó sẽ nhận diện sai hàng loạt các hình số 4 tương tự trong tương lai.



Hình 5.4: Dữ liệu gán nhãn sai trong MNIST

Sự phình to của Kênh Nhiễu (Noise Channels)

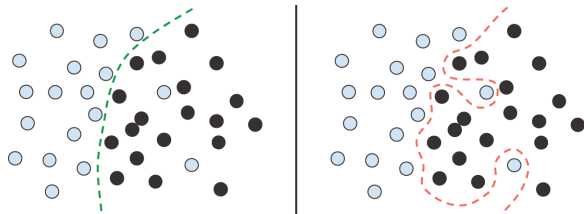
- Thí nghiệm: Thêm một số ma trận toàn nhiễu trắng (white noise) hoặc toàn số 0 nối thêm vào dữ liệu MNIST ban đầu.
- Kết quả: Kể cả khi dữ liệu MNIST hợp lệ vẫn nguyên vẹn, mạng bị phân tâm bởi kênh nhiễu.
- Validation Accuracy của bộ dữ liệu chứa Noise Channels giảm rõ rệt. Lý do: Mô hình vô tình tìm ra các mẫu (patterns) ngẫu nhiên giả mạo trong tập nhiễu và sử dụng chúng để ra quyết định → Suy giảm Generalization.



Hình 5.5: Tác động của kênh nhiễu đến Accuracy

Ngoại lai (Outliers) và Sự quá khớp

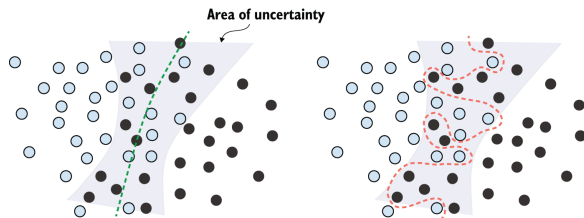
- Các điểm nhiễu thường đóng vai trò là "Ngoại lai" (Outliers), tức là các điểm dữ liệu nằm lạc lõng giữa quần thể lớp khác.
- Việc Gradient Descent nỗ lực cực độ để phân loại đúng 100% tập huấn luyện sẽ ép đường ranh giới quyết định vươn xa ra ngoài để bọc lấy outlier.
- Hậu quả: Ranh giới quyết định trở nên góc cạnh thay vì là một đường cong mượt mà, bao quát.



Hình 5.6: Robust fit (trái) vs. Overfitting ngoại lai (phải)

Đặc trưng mơ hồ (Ambiguous features)

- Đôi khi ranh giới giữa hai nhóm dữ liệu có vùng giao thoa. Tại vùng này, một dữ liệu có xác suất 50% thuộc lớp A và 50% thuộc lớp B (Uncertainty).
- Thay vì vạch một đường thẳng cắt đôi đám giao thoa mang tính chất thống kê này, Overfitting sẽ biến đường biên thành hình răng cưa uốn éo cố phân chia tách bạch.
- Khắc phục: Phải cho phép mô hình phạm lỗi trên tập Train (Training Loss không bao giờ bằng 0).



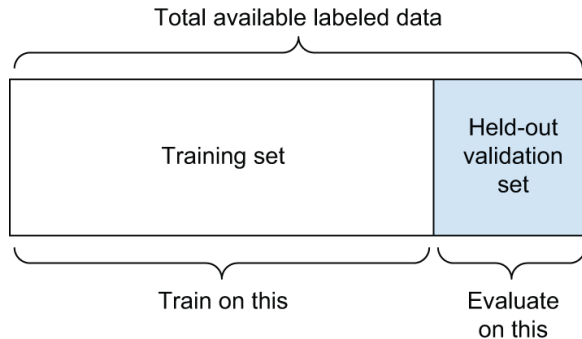
Hình 5.7: Overfitting trên vùng dữ liệu bất định

Chia 3 tập: Train, Validation và Test

- Tại sao không chỉ có 2 tập Train và Test?
- Quá trình huấn luyện không chỉ là điều chỉnh "Tham số" (Weights/Biases) qua Gradient Descent mà còn là điều chỉnh "Siêu tham số" (Hyperparameters - số lớp ẩn, số nơ-ron, learning rate...).
- Chúng ta dò tìm các cấu hình siêu tham số này để cải thiện điểm số trên tập Validation.
- **Information Leak:** Việc liên tục dùng tập Validation làm la bàn để tối ưu siêu tham số sẽ khiến mô hình gián tiếp "học lỏm" cấu trúc tập Validation → Overfit trên chính Validation.
- Do đó, ta bắt buộc phải có tập thứ 3: **Tập Test** (Hoàn toàn mới, chỉ chạy đúng 1 lần trước khi đưa ứng dụng vào thực tế).

Tách tập Validation đơn giản (Hold-out Validation)

- Xáo trộn dữ liệu (Shuffle) và tách một phần cố định (vd: 20%) từ tập Training để làm tập Validation.
- Phương pháp này rất nhanh gọn, thường dùng khi khối lượng dữ liệu khổng lồ (hàng trăm ngàn điểm dữ liệu).
- Nhược điểm: Nếu dữ liệu ít, tập Validation sẽ không mang tính đại diện thống kê cho toàn bộ phân phối, dẫn tới sai số đánh giá lớn khi đổi seed chia tách.



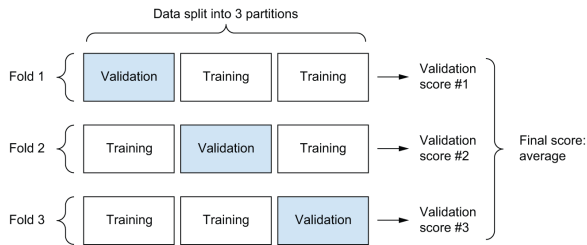
Hình 5.8: Quy trình Hold-out Validation

Mã nguồn: Hold-out Validation

```
1 import numpy as np
2
3 num_val_samples = 10000
4 np.random.shuffle(data) # Xao tron ngau nhien
5
6 # Tach 10k mau lam Validation
7 val_data = data[:num_val_samples]
8 train_data = data[num_val_samples:]
9
10 model = get_model()
11 model.fit(train_data, epochs=20)
12 val_score = model.evaluate(val_data)
13
14 # Sau khi dieu chinh hyperparameter thanh cong,
15 # ta huan luyen lai 1 mo hinh tren toan bo Train + Validation
16 model = get_model()
17 model.fit(np.concatenate([train_data, val_data]), epochs=20)
18 test_score = model.evaluate(test_data)
```

Đánh giá chéo K-Lần (K-Fold Validation)

- Khi dữ liệu cực kỳ thưa thớt, việc Hold-out 1/5 lượng dữ liệu sẽ làm lãng phí cơ hội học tập của mạng.
- **K-Fold Validation:** Chia dữ liệu thành K phần (Fold). Lặp lại K vòng huấn luyện khác nhau. Ở vòng i , lấy Fold i làm Validation, gộp tất cả $K - 1$ Fold còn lại làm Training.
- Điểm số cuối cùng là giá trị trung bình của K điểm số đánh giá.
- Độ chính xác cao, nhưng chi phí tài nguyên gấp K lần.



Hình 5.9: Quy trình K-Fold Validation (với $K=3$)

Mã nguồn: K-Fold Validation

```
1 k = 4
2 num_val_samples = len(data) // k
3 val_scores = []
4
5 for fold in range(k):
6     # Lay ra phan Validation rieng
7     val_data = data[num_val_samples * fold : num_val_samples * (fold + 1)]
8
9     # Gop phan dau va cuoi de tao thanh tap Train
10    train_data = np.concatenate(
11        [data[: num_val_samples * fold], data[num_val_samples * (fold + 1) :]], axis
12        =0
13    )
14
15    model = get_model() # Khoi tao lai mo hinh moi hoan toan!
16    model.fit(train_data, epochs=100, batch_size=16)
17    score = model.evaluate(val_data)
18    val_scores.append(score)
19
20 average_val_score = np.mean(val_scores) # Lay trung binh
```